VoiceOps 2026 — ZentroMart voice support agent

Sreesha (individual submission) | Code: https://github.com/Sreesha-S/VoiceOps_Project

Summary. I built a customer-support agent for the fictional store ZentroMart in four steps. It answers from the company documents, remembers the conversation, verifies the caller before sharing order details, hands over to a human through a real ticket, and talks over audio with Gnani speech models. On the 52 practice questions the document-grounded agent passed **50/52 (96.2%)** versus **18/52 (34.6%)** for the prompt-only baseline. On the 23 practice conversations (96 customer turns) it passed **92/96 turns (95.8%)**. These numbers come from the same practice data I used while developing, so the hidden test score will probably be lower.

Pipeline for one spoken turn: audio → Gnani STT → code reads order ID / verification digits → Gemini rewrites the message into a clean English search query → hybrid search (keyword + Gemini embeddings) → Gemini answers only from the retrieved passages (and may call date, pincode and ticket tools) → speech-friendly cleanup → Gnani TTS → audio.

1. What I built in each step

- **Step 1 - prompt only.** A ZentroMart system prompt (role, limits, AI disclosure in the first reply, no invented order details). It has not seen any document, so it guesses. This is the baseline.
- **Step 2 - RAG.** One chunk per document section (section ID, document type, status and effective date kept) plus one chunk per product row. Search = BM25 keyword scores + Gemini embeddings merged with reciprocal rank fusion, then weights for the precedence rule (policy > agent guide > help > FAQ > marketing; superseded documents last). Products have their own capped search lane. The model must answer only from the retrieved text, say "I don't have that information" and offer a human when the documents are silent, and every answer logs the chunk IDs used. A date tool (business days skip Sundays and holidays.csv, using the fixed date 20 Oct 2026) replaces mental arithmetic.
- **Step 3 - conversation.** Per-call memory; follow-ups reuse the order already discussed. Identity checks are done by *code*, not by the model: order IDs and the 4-digit phone suffix or 6-digit pincode are read from the customer's words (typed, spoken or Devanagari digits), lookup_order is called, one retry is allowed and then the call is locked. The model sees order facts only after verification and never sees the secrets; saved logs mask them. create_ticket is wrapped so secrets are scrubbed from the summary, and a guard re-asks the model if a reply claims a ticket that does not exist.
- **Step 4 - voice.** Gnani STT (en-IN,hi-IN code-switching) and Gnani TTS (timbre-v2.5, voice Kaveri, speed 0.95). Unclear or empty audio gets a polite "could you say it again?"; Gnani errors are retried and never crash the run; replies are normalised for the ear (order numbers in groups of four digits, dates with the year, rupees spelled out, no markdown). run.py processes a folder of audio files and saves transcript, reply text, reply audio and the document chunks used.

2. Results (practice data, final code; both steps on Gemini gemini-3.5-flash-lite)

Practice questions (52)	Step 1: no documents	Step 2: with documents
Passed (must_include / must_not_include)	18 / 52 (34.6%)	50 / 52 (96.2%)
English / Hindi / Hinglish	39.4% / 25.0% / 27.3%	93.9% / 100% / 100%
Answerable / not in the documents	34.0% / 40.0%	95.7% / 100%
Retrieval: mean recall of gold sections	not applicable	0.954 (all gold sections found for 89.4% of questions)

Practice conversations (23 conversations, 96 turns)	Result
Turns passed (facts, forbidden phrases, expected tool calls)	92 / 96 (95.8%); 19 of 23 conversations fully passed
By expected decision	clarify 100%, escalate 100% (ticket really created), answer 94.4%, refuse 66.7%
Retrieval: mean recall of gold sections	0.967

What the comparison shows. Without documents the model cannot know ZentroMart's windows, fees, timelines or the Wallet/Plus rules, so it passes mostly the "I don't know"-type questions by luck; Hindi and

Hinglish are worst because it also guesses in a second language. Grounding in the documents more than doubled correctness, and every unanswerable question was answered with an honest "I don't have that information".

Honesty notes. (1) Escalation and privacy rules (KB-00.1, KB-00.3, ESC-*, IDV-4) are always shown to the model and counted as used chunks, which raises recall. (2) The same 52 questions guided my prompt and search fixes, so 96% is optimistic. (3) The checker is string-based (it also accepts digits for number words), so it can miss a correct paraphrase or accept a lucky phrase. (4) Voice was tested by hand, not scored automatically.

3. How the score improved (Step 2 on the 52 questions)

Run	Passed	What changed / what I learned
First RAG run	40/52 (77%)	5 answers were empty (Gemini returned nothing and my cache stored the empty answer); search missed rules such as innerwear and opened beauty items.
After fixes 1	48/52 (92%)	Retry on empty answers and never cache them; retrieve 8 chunks instead of 5; richer query-rewrite vocabulary; escalation rules always shown; number words counted as digits.
Weaker model test	43/52 (83%)	Same code on gemini-3.1-flash-lite: refund dates off by 1-2 days, an invented "we do not offer midnight delivery". The model matters a lot.
Final	50/52 (96%)	business_day_range tool (one call gives "between X and Y"), product-category-aware search (briefs are innerwear), digits not words, rule against inventing negatives, refusals start with "Sorry", ticket honesty guard.

4. What went wrong and what I would fix next

- **Date arithmetic.** Models chained two date calls and slipped by a day. Fixed with one tool call; the tool is unit-tested against all refund-date practice answers.
- **Claiming actions that did not happen.** One reply said a ticket was raised when none existed. A guard now forces a real create_ticket (or removes the claim). The code, not the model, decides whether a ticket exists.
- **Escalation by price.** A cracked laptop (above the 10,000 rupee rule) was not handed over because the model did not compare price with the threshold. The program now reads the threshold from ESC-1, compares it with the catalog price and tells the model.
- **Speech recognition errors (real recordings).** Gnani heard "straight kurta" as "street kurta", "sale" as "same" and wrote numbers as words ("fourteen", "five october"). The agent still answered sensibly because search uses meaning as well as keywords, but a misheard product name is a real risk. One Gnani request returned HTTP 504 and the automatic retry recovered.
- **Voice quality.** Replies sounded slightly rushed and Hindi words written in Latin letters ("madad") were read by an English voice. I added a speed setting (0.95) and a voice_test.py script to compare voices; a Hindi-capable voice for Hinglish replies is the next improvement.
- **Still failing (not fully analysed).** Two practice questions (PQ0025, PQ0050) and four conversation turns (PC004 T4, PC017 T4, PC019 T4, PC020 T4; three are refusals about cancelling a shipped order). The likely cause is a missing required detail (for example the return window in days) rather than a wrong policy; I would print each failing row and add the missing fact to the answer rules.
- **Other limits.** Clips over 60 seconds fail on Gnani REST (I would split long audio); I did not use streaming STT/TTS or barge-in; evaluation is string-based, an LLM judge would be fairer; free-tier Gemini limits (about 500 requests per day on lite models) shaped how often I could re-run, so I added request pacing and an on-disk cache.

5. Bonus work

- Code-enforced caller verification with lock-out, secrets masked in logs, ticket summaries and replies.
- 27 offline unit tests (no keys needed): chunking, search rules, date tool, number reading in four forms, verification and lock-out, ticket honesty, voice fallbacks with fake Gnani clients, tool-calling loop.
- check_setup.py (tests every key and service step by step), evaluate_questions.py and evaluate_conversations.py (reproduce every number above), voice_chat.py for live voice, make_submission.py (builds the zip and checks that no secret is inside).